

# DSA & Advanced DSA Roadmap

## Phase 3 — Linked Lists

### Learning Objectives

#### By the end of this phase, learners will be able to:

- Explain how linked lists differ from arrays in memory layout and access cost.
- Implement singly, doubly, and circular linked lists from scratch, including node design and pointer manipulation.
- Use the dummy node technique to simplify edge cases in list manipulation.
- Identify and apply core linked list patterns: fast & slow pointers, in-place reversal, merging sorted lists, and cycle detection.
- Solve Easy to Medium level linked list problems independently.

### Implementation & Internals

#### Core Concepts

- Singly Linked List — each node holds a value and a pointer to the next node.
- Doubly Linked List — each node also holds a pointer to the previous node, enabling  $O(1)$  backward traversal.
- Circular Linked List — the last node points back to the first, useful for round-robin style structures.
- Node Design — representing a node as a small class/struct bundling value and pointer(s).
- Pointer Manipulation — the core skill of this phase: carefully rewiring next (and prev) pointers without losing access to the rest of the list.

#### Implementation Exercises

- Build a singly linked list from scratch supporting insert-at-head, insert-at-tail, delete-by-value, and traversal/print.
- Extend the above into a doubly linked list, adding a prev pointer and supporting backward traversal.
- Build a circular linked list and implement a function that detects whether traversal has looped back to the start.
- Implement a generic dummy-head wrapper and use it to simplify head-removal edge cases in your own delete function.

### Pattern Mastery

#### Pattern 1: Fast & Slow Pointer

##### Concepts

- Two pointers moving through the list at different speeds (typically 1x and 2x).
- Used to find the middle of a list, detect cycles, and find nodes a fixed distance from the end.

##### Practice Problems

- Find the middle node of a linked list.
- Remove the nth node from the end of a list using a single pass.

##### LeetCode

- [Middle of the Linked List — leetcode.com/problems/middle-of-the-linked-list](https://leetcode.com/problems/middle-of-the-linked-list)
- [Remove Nth Node From End of List — leetcode.com/problems/remove-nth-node-from-end-of-list](https://leetcode.com/problems/remove-nth-node-from-end-of-list)

## Pattern 2: Reverse Linked List

### Concepts

- Iterative in-place reversal using three pointers (prev, current, next).
- Recursive reversal, and the stack-depth cost of doing it that way.
- Reversing only a sub-section of a list between two given positions.

### Practice Problems

- Reverse an entire singly linked list.
- Reverse only the nodes between positions left and right.
- Check whether a linked list is a palindrome by reversing the second half.

### LeetCode

- [Reverse Linked List — leetcode.com/problems/reverse-linked-list](https://leetcode.com/problems/reverse-linked-list)
- [Reverse Linked List II — leetcode.com/problems/reverse-linked-list-ii](https://leetcode.com/problems/reverse-linked-list-ii)
- [Palindrome Linked List — leetcode.com/problems/palindrome-linked-list](https://leetcode.com/problems/palindrome-linked-list)

## Pattern 3: Merge Lists

### Concepts

- Merging two sorted lists by repeatedly choosing the smaller head and rewiring pointers.
- Extending pairwise merging to k lists using a min-heap.
- Why splicing existing nodes (not creating new ones) keeps the merge  $O(1)$  extra space for two lists.

### Practice Problems

- Merge two sorted linked lists into one sorted list.
- Merge k sorted linked lists into one sorted list.

### LeetCode

- [Merge Two Sorted Lists — leetcode.com/problems/merge-two-sorted-lists](https://leetcode.com/problems/merge-two-sorted-lists)
- [Merge k Sorted Lists — leetcode.com/problems/merge-k-sorted-lists](https://leetcode.com/problems/merge-k-sorted-lists)

*Note: Merge k Sorted Lists is rated Hard and depends on the Heap pattern from Phase 12. Students can attempt it here with a min-heap explained inline, then revisit it after Phase 12 for a deeper treatment.*

## Pattern 4: Cycle Detection

### Concepts

- Floyd's Tortoise and Hare algorithm: a slow and fast pointer that must eventually meet if a cycle exists.
- Proving why the pointers meet inside the cycle, and how to find the cycle's starting node once they do.

### Practice Problems

- Determine whether a linked list contains a cycle.
- Find the node where the cycle begins, if one exists.

### LeetCode

- [Linked List Cycle — leetcode.com/problems/linked-list-cycle](https://leetcode.com/problems/linked-list-cycle)
- [Linked List Cycle II — leetcode.com/problems/linked-list-cycle-ii](https://leetcode.com/problems/linked-list-cycle-ii)

## Pattern 5: Dummy Node Pattern

### Concepts

- Introducing a sentinel node before the head to eliminate special-casing the head during insertion or deletion.
- Returning dummy.next at the end instead of tracking the real head separately.

### Practice Problems

- Remove all nodes matching a given value, including possibly the head.
- Remove duplicate values from a sorted linked list.

### LeetCode

- [Remove Linked List Elements](https://leetcode.com/problems/remove-linked-list-elements) — [leetcode.com/problems/remove-linked-list-elements](https://leetcode.com/problems/remove-linked-list-elements)
- [Remove Duplicates from Sorted List](https://leetcode.com/problems/remove-duplicates-from-sorted-list) — [leetcode.com/problems/remove-duplicates-from-sorted-list](https://leetcode.com/problems/remove-duplicates-from-sorted-list)

## Phase Assessment

### Implementation Tasks

- Build a singly linked list from scratch with insert, delete, and traversal.
- Implement iterative reversal of a singly linked list.
- Analyze and explain why fast & slow pointers solve middle-finding and cycle detection in  $O(n)$  time and  $O(1)$  space.

### Recommended LeetCode Target

#### Easy Problems

- [leetcode.com/problems/reverse-linked-list](https://leetcode.com/problems/reverse-linked-list)
- [leetcode.com/problems/middle-of-the-linked-list](https://leetcode.com/problems/middle-of-the-linked-list)
- [leetcode.com/problems/merge-two-sorted-lists](https://leetcode.com/problems/merge-two-sorted-lists)
- [leetcode.com/problems/linked-list-cycle](https://leetcode.com/problems/linked-list-cycle)
- [leetcode.com/problems/palindrome-linked-list](https://leetcode.com/problems/palindrome-linked-list)
- [leetcode.com/problems/remove-linked-list-elements](https://leetcode.com/problems/remove-linked-list-elements)
- [leetcode.com/problems/remove-duplicates-from-sorted-list](https://leetcode.com/problems/remove-duplicates-from-sorted-list)

#### Medium Problems

#### Fast & Slow Pointer

- [leetcode.com/problems/remove-nth-node-from-end-of-list](https://leetcode.com/problems/remove-nth-node-from-end-of-list)

#### Reverse Linked List

- [leetcode.com/problems/reverse-linked-list-ii](https://leetcode.com/problems/reverse-linked-list-ii)

#### Cycle Detection

- [leetcode.com/problems/linked-list-cycle-ii](https://leetcode.com/problems/linked-list-cycle-ii)

#### Hard Problems

- [leetcode.com/problems/merge-k-sorted-lists](https://leetcode.com/problems/merge-k-sorted-lists)

#### On scope

*Merge k Sorted Lists is the only Hard problem this phase, and it previews the Heap pattern (Phase 12) ahead of schedule. This is intentional: it can be solved here with a min-heap explained as a black box ('always gives you the smallest of the current candidates'), then revisited for a full treatment once heaps are formally covered.*

### Coding Challenges (Assessment Day)

- Reverse Linked List
- Merge Two Sorted Lists

- Linked List Cycle II
- Remove Nth Node From End of List
- Palindrome Linked List

## Expected Outcome

### Students should be able to:

- Implement singly and doubly linked lists from scratch, including insertion, deletion, and traversal.
- Recognize common linked list patterns: fast & slow pointers, reversal, merging, cycle detection, and the dummy node technique.
- Solve unseen linked list problems using pattern-based thinking rather than memorized solutions.
- Complete Easy and Medium linked list questions independently, and approach Hard problems with a guided pattern (e.g. heap-based merging) even before that pattern is formally taught.